



Estruturas de Repositórios, Branches, Tags e Submódulos

Gerenciamento avançado de código com Git para colaboração e versionamento eficaz.

INSTITUIÇÃO EXECUTORA



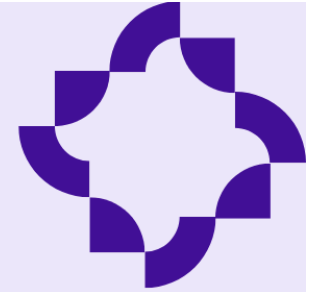
COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Sumário

- Arquitetura de Repositórios Git e Componentes
- Branches: Conceitos, Operações e Estratégias
- Tags: Versionamento Semântico e Imutabilidade
- Submódulos: Dependências e Gerenciamento de Projetos



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Estruturas de Repositórios Modernas



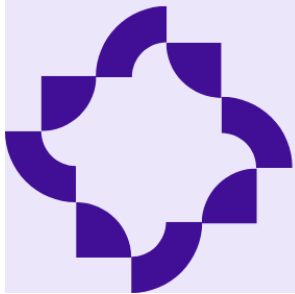
Working Directory

O *Working Directory* é a área onde os arquivos do projeto são editados fisicamente no sistema de arquivos local. Reflete o estado atual dos arquivos que estão sendo modificados ou desenvolvidos ativamente pelo usuário.



Staging Area (Index)

A *Staging Area*, também conhecida como Index, é um intermediário crucial. Ela prepara as mudanças que serão incluídas no próximo commit, permitindo que os desenvolvedores selecionem quais modificações serão registradas no histórico do repositório.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Estruturas de Repositórios Modernas



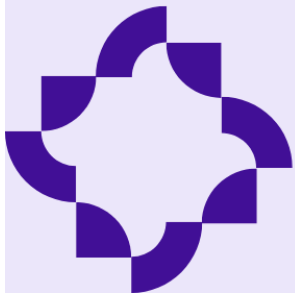
Repositório Local

O **Repositório Local** contém todo o histórico do projeto, incluindo todos os commits e branches. É um banco de dados de objetos que rastreia mudanças e permite acesso offline a todas as versões do código.



Repositório Remoto

O **Repositório Remoto** é um servidor central (e.g., GitHub, GitLab, Bitbucket) que hospeda uma cópia do repositório do projeto. Facilita a colaboração entre equipes, permitindo que múltiplos desenvolvedores compartilhem e sincronizem seus trabalhos.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Branches: Conceitos e Isolamento

- **Ponteiros Leves:** Criar um branch no Git é instantâneo, utilizando apenas 41 bytes para apontar para um commit específico.
- **HEAD:** Este é um ponteiro especial que serve como indicador do branch atualmente ativo no seu diretório de trabalho.
- **Paralelismo:** Permite que diferentes funcionalidades, correções de bugs e experimentos sejam desenvolvidos simultaneamente sem conflitos diretos.
- **Independência:** Alterações realizadas em um branch permanecem isoladas e não afetam outros branches até que seja realizado um merge ou rebase.

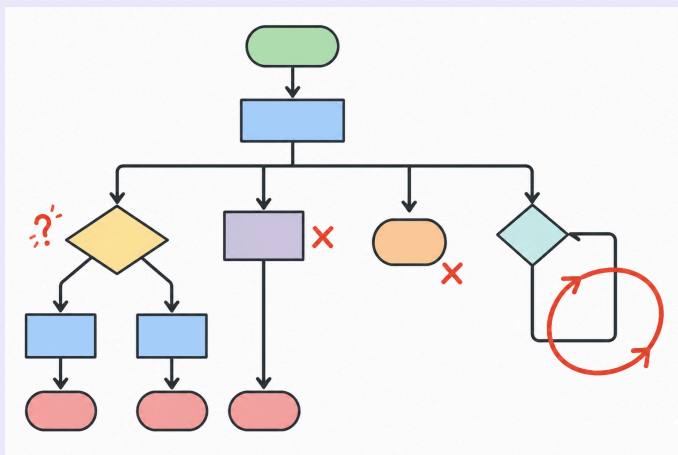
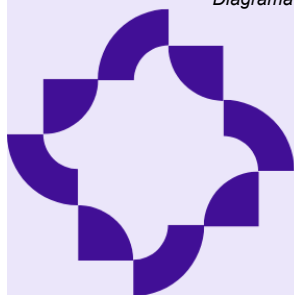


Diagrama de fluxo ilustrando caminhos, decisões e loops de processo.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Ciclo de Vida de Branches

1

Criar e Navegar Branches

O ciclo inicia-se com a criação de um novo branch, que é essencialmente um **ponteiro** para um commit específico. Utilize **git branch** novo-branch para criar o ponteiro e, em seguida, navegue para ele com **git checkout** novo-branch ou o comando mais moderno **git switch** novo-branch para alterar o contexto do diretório de trabalho.

2

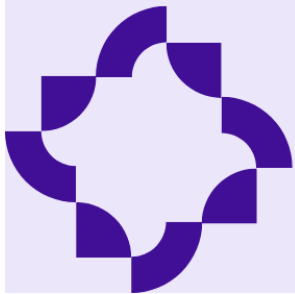
Realizar Fusão (Merging)

Após o desenvolvimento isolado, os branches precisam ser integrados. A **fusão (merging)** une as alterações de um branch em outro. Existem dois tipos principais: **Fast-forward**, que ocorre quando o branch de destino não possui novos commits, simplesmente avançando o ponteiro; e **Three-way Merge**, que cria um novo commit de merge para reconciliar histórias que divergiram, mantendo um registro explícito da união.

3

Excluir Branches

Uma vez que as alterações de um branch são integradas com sucesso e o branch não é mais necessário para desenvolvimento ativo, ele deve ser excluído. Esta etapa é crucial para manter o repositório organizado e limpo, removendo referências a branches concluídos e focando nos ativos.



INSTITUIÇÃO EXECUTORA



COORDENADORA

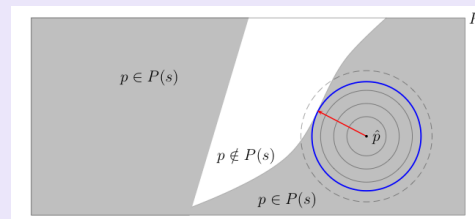


APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Estratégia Gitflow

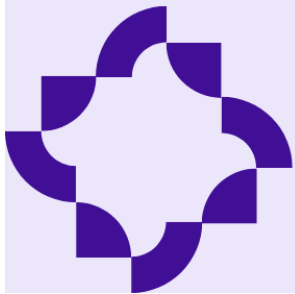


Branch Main

A branch **Main** representa o código em **produção**, sendo sempre estável e pronta para implantação. É a referência do que está ativo para os usuários.

Branch Develop

A branch **Develop** é a base para o próximo *release*. Nela são integradas todas as *features* finalizadas de diversas branches, aguardando testes e validação.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



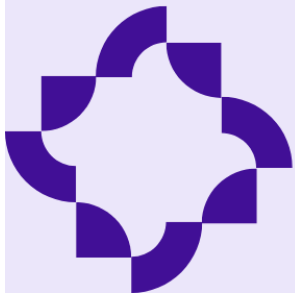
Estratégia Gitflow

Feature Branches

As **Feature Branches** são criadas a partir de ``develop`` para desenvolver *features* específicas de forma isolada. Uma vez concluídas, são mergeadas de volta em ``develop``.

Release Branches

As **Release Branches** são criadas a partir de ``develop`` para preparar o próximo lançamento. Nelas são feitas correções finais de bugs e atualizações de documentação, antes de serem mergeadas em ``main`` e ``develop``.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

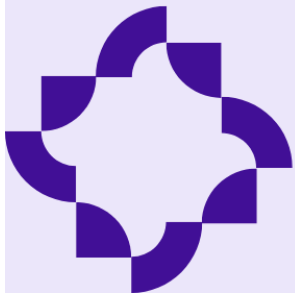




Estratégia Gitflow

Hotfix Branches

As **Hotfix Branches** são essenciais para lidar com mudanças críticas diretamente em produção. Criadas a partir de 'main', elas são mergeadas em 'main' e 'develop' após a correção para garantir a portabilidade.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

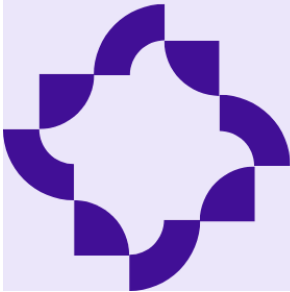




Trunk-Based Development

A tabela compara as características principais entre o Gitflow e o Trunk-Based Development.

Característica	Gitflow	Trunk-Based
Integração	Longo prazo em branches de feature	Diária no branch principal
Complexidade	Alta (múltiplos tipos de branch)	Baixa (foco no trunk)
Code Review	No final da feature	Pequenas e frequentes
Risco de Conflito	Maior no merge final	Menor devido a commits pequenos
Ideal para	Ciclos de release tradicionais	CI/CD e entregas rápidas



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



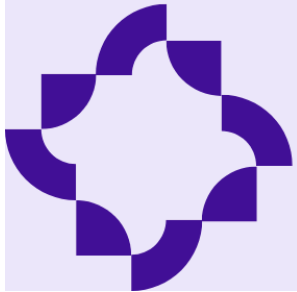


Tags Leves

São meramente um **ponteiro** para um commit específico, equivalente a um *branch* que não avança. Úteis para referências temporárias ou locais, não armazenam metadados adicionais do *tagger*.

Tags Anotadas

São objetos completos no banco de dados do Git, contendo nome, e-mail, data e mensagem do *tagger*. Podem ser **assinadas digitalmente**, sendo ideais para *releases* oficiais e permanentes.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Versionamento Semântico (SemVer)

Versão MAJOR

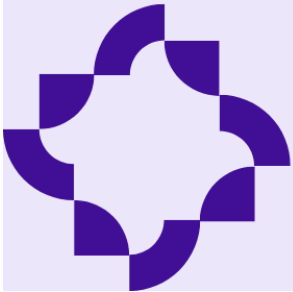
Incrementada quando há mudanças **incompatíveis** na API pública (quebra de contrato), exigindo que os consumidores atualizem seu código para manter a compatibilidade. Indica uma evolução significativa com impacto.

Versão MINOR

Incrementada quando se adiciona funcionalidade de maneira **retrocompatível**, sem quebrar a API existente. Novas funcionalidades são introduzidas sem a necessidade de alterações no código dos usuários.

Versão PATCH

Incrementada quando se fazem correções de *bugs* de maneira **retrocompatível**. Não introduz novas funcionalidades nem quebra a API, focando apenas em melhorias e correções internas.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

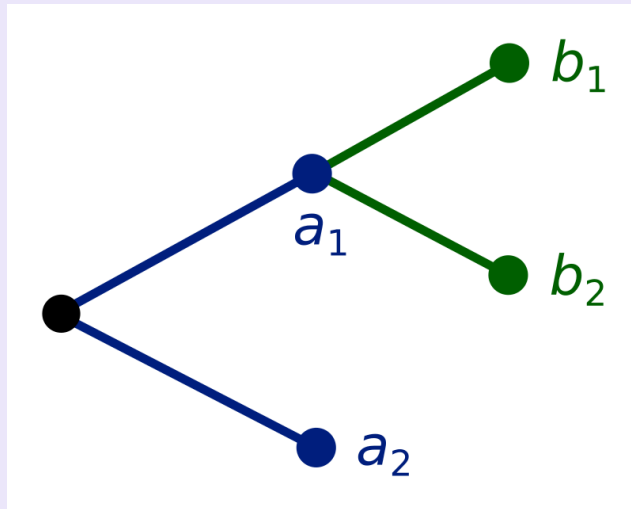


MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

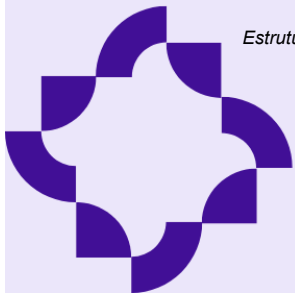




Submódulos: Dependências Externas



- **Integração Flexível:** Mantém um repositório Git como subdiretório de outro, ideal para consumo de bibliotecas ou módulos de terceiros.
- **Modularidade** do Projeto: Permite dividir sistemas monolíticos em componentes menores e gerenciáveis, promovendo reusabilidade.
- **Versionamento Independente:** O repositório principal referencia um commit específico do submódulo, isolando-o de atualizações externas.
- **Gestão de Dependências:** Facilita a gestão de múltiplas dependências externas dentro de um projeto principal, mantendo-as separadas e rastreáveis.



Estrutura de árvore com três folhas, representando submódulos.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO





Fluxo de Trabalho de Submódulos

1

Adicionar Submódulo

Utilize `git submodule add <url>` para registrar um repositório externo como submódulo.

O repositório é referenciado no arquivo `.gitmodules` e integrado ao projeto principal.

2

Clonar Submódulos

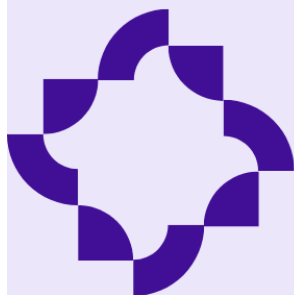
Ao clonar o projeto principal, os submódulos ficam vazios.

Execute `git submodule update --init` para inicializá-los e baixar seus conteúdos.

3

Atualizar Submódulos

Para incorporar as últimas alterações do repositório remoto de um submódulo ao seu projeto, utilize o comando de atualização apropriado para sincronizar o HEAD do submódulo.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

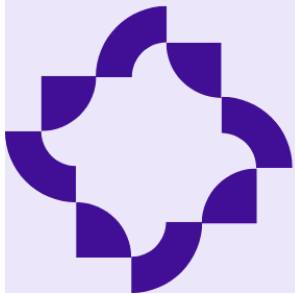


Submódulos

Mantêm apenas um **link** (hash) para o repositório externo, sem armazenar o conteúdo diretamente no repositório pai. Ideal para o gerenciamento de *dependências* ou bibliotecas de terceiros, garantindo estrito isolamento e desacoplamento. Requer comandos de inicialização específicos para clonar o conteúdo.

Subtrees

Copiam fisicamente o **código** do repositório externo para o pai, integrando-o diretamente. Mais simples para outros desenvolvedores, pois dispensa comandos especiais de *init*. No entanto, torna o histórico do repositório principal mais denso devido ao histórico do repositório aninhado.



INSTITUIÇÃO EXECUTORA



COORDENADORA



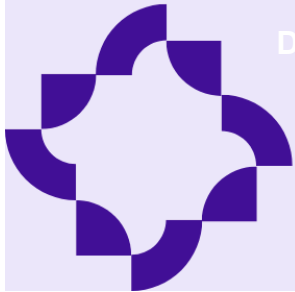
APOIO





Um desenvolvedor precisa marcar a versão final de um produto estável que será enviada para produção, garantindo que a informação do autor e a data sejam preservadas no marcador. Qual recurso do Git deve ser utilizado para esse propósito?

- A Um Submódulo apontando para o commit de release.
- B Um branch chamado 'production'.
- C Uma Tag Leve (Lightweight Tag).
- D Uma Tag Anotada (Annotated Tag).



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



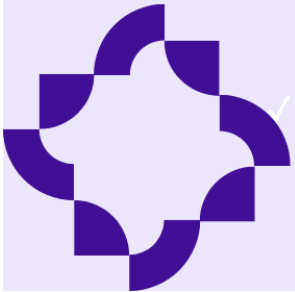
Um desenvolvedor precisa marcar a versão final de um produto estável que será enviada para produção, garantindo que a informação do autor e a data sejam preservadas no marcador. Qual recurso do Git deve ser utilizado para esse propósito?

✗ Um Submódulo apontando para o commit de release.

✗ Um branch chamado 'production'.

✗ Uma Tag Leve (Lightweight Tag).

✓ Uma Tag Anotada (Annotated Tag).



INSTITUIÇÃO EXECUTORA



COORDENADORA



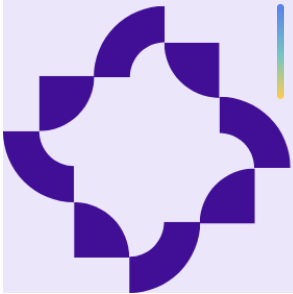
APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Discuta com o colega ao lado

Considerando uma startup com microsserviços e múltiplos *deploys* diários, qual a melhor estratégia de *branching* e como gerenciar os *submódulos* para evitar débito técnico?



Discuta como as escolhas de branching (Gitflow vs. Trunk-based) e o uso de submódulos impactam a consistência e a velocidade da equipe.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO





Conclusão

- Branches facilitam desenvolvimento paralelo, usando ponteiros leves e operações como merge ou rebase para integração.
- Estratégias como Gitflow ou Trunk-Based Development otimizam fluxos, equilibrando estrutura com agilidade na entrega.
- Tags, especialmente anotadas, são marcos imutáveis para releases, aderindo ao Versionamento Semântico (MAJOR.MINOR.PATCH).
- Submódulos gerenciam dependências externas, linkando repositórios, mas exigem atenção à inicialização e atualização do hash.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO